Computer Organization & Assembly Language

Lecture 1

Outline

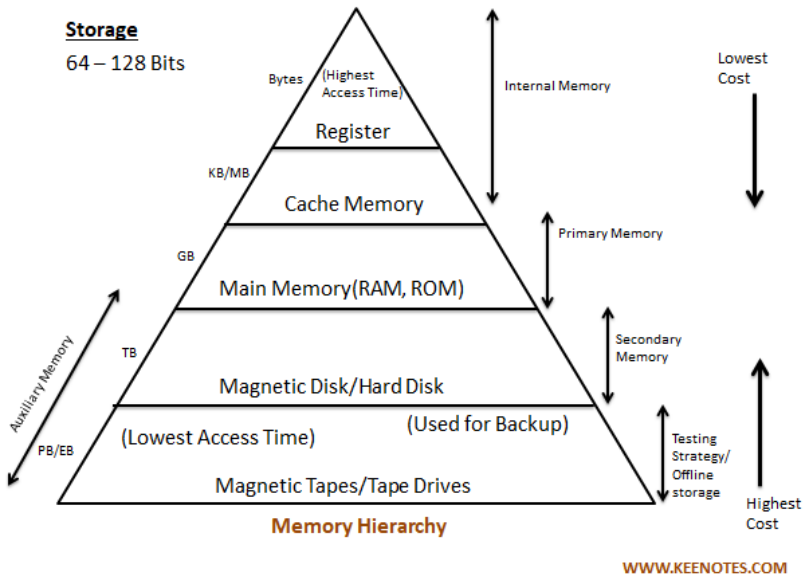
- About this Course
- Basic Structure of Computer
- What is Computer Organization?
- About Assembly Language

Course Objectives

- To learn the organization of computer hardware components.
- To gain knowledge about the internal architecture and working of microprocessors.
- To understand working of memory devices, interrupt controllers and I/O devices.
- Develop skills to analyze and optimize performance at the machine level.
- To understand syntax, semantics, and structure of assembly language.

Course Contents

- Computer Architecture
- Assembly Language Fundamentals
- Data Representation
- Memory Hierarchy
- Input/Output Systems



Real World Applications

- Embedded Systems
  - Home Automation (Smart Homes)
  - Automotive Systems (Vehicles)
- Missile Guidance Systems
- Home Appliances
- Medical Devices
- Consumer Electronics (Smartphones)
- Robotics

Why Assembly Language?

- Direct Hardware Control
- Performance Optimization
- Understanding Computer Architecture
- Embedded Systems Expertise (low cost)
- Real-Time Processing
- Foundation for Other Languages

Structure

- **Structure** is the way in which components relate to each other

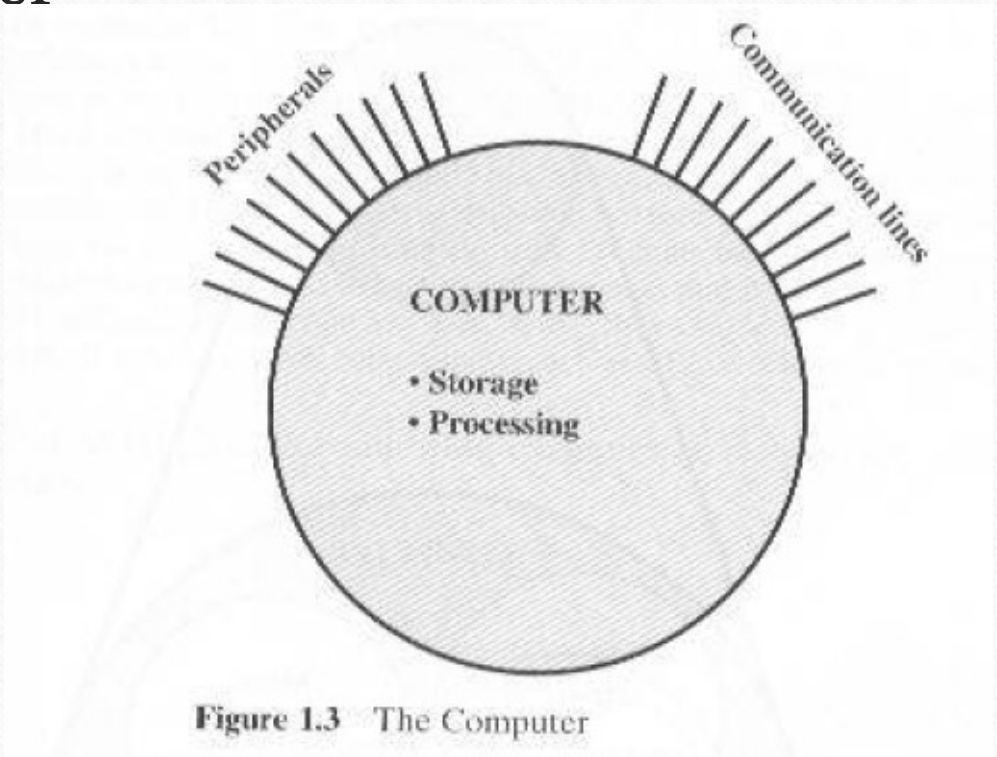


Figure 1.3 The Computer

Difference in Peripherals & Communication Lines

- When data is received from or delivered by a device that is directly connected to the computer, process is called **Input-Output** (I/O).
- When data are moved over longer distance, to or from a remote device, the process is known as **Data Communication**.

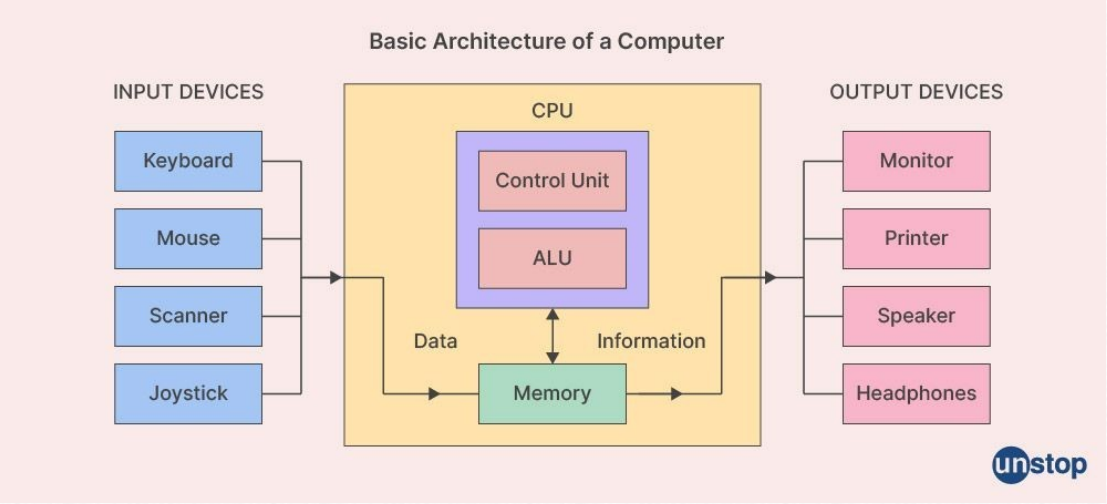
Computer Architecture

- Computer Architecture refers to those attributes of a system visible to a programmer
  - Those attributes that have direct impact on logical execution of a program.
- **Architectural attributes include**:
  - the instruction set,
  - no. of bits used to represent various data types (numbers, characters etc),
  - I/O mechanisms and technology for addressing memory.

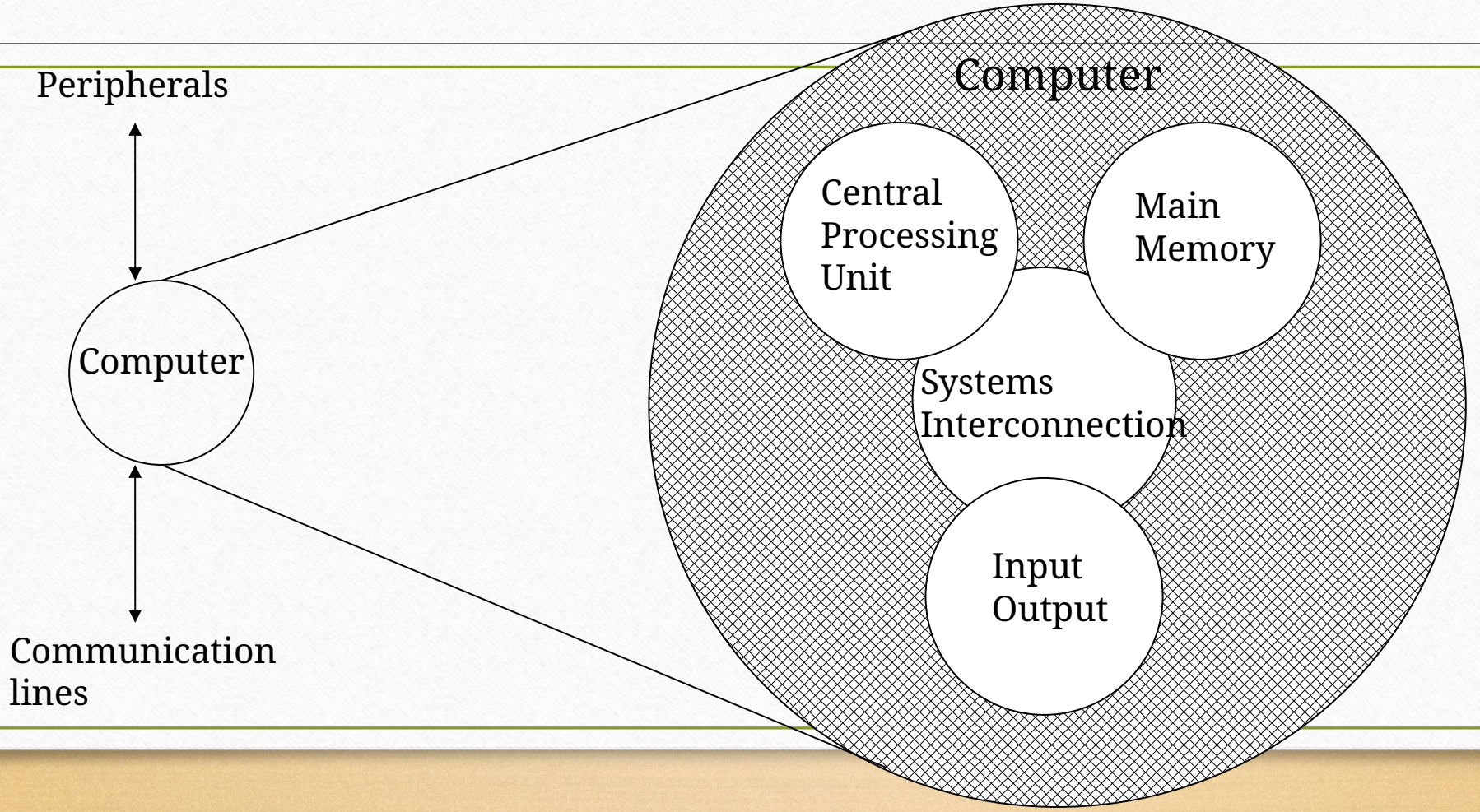
- **Example**: Architectural design issue whether a computer will have multiple instruction or not.

Computer Architecture

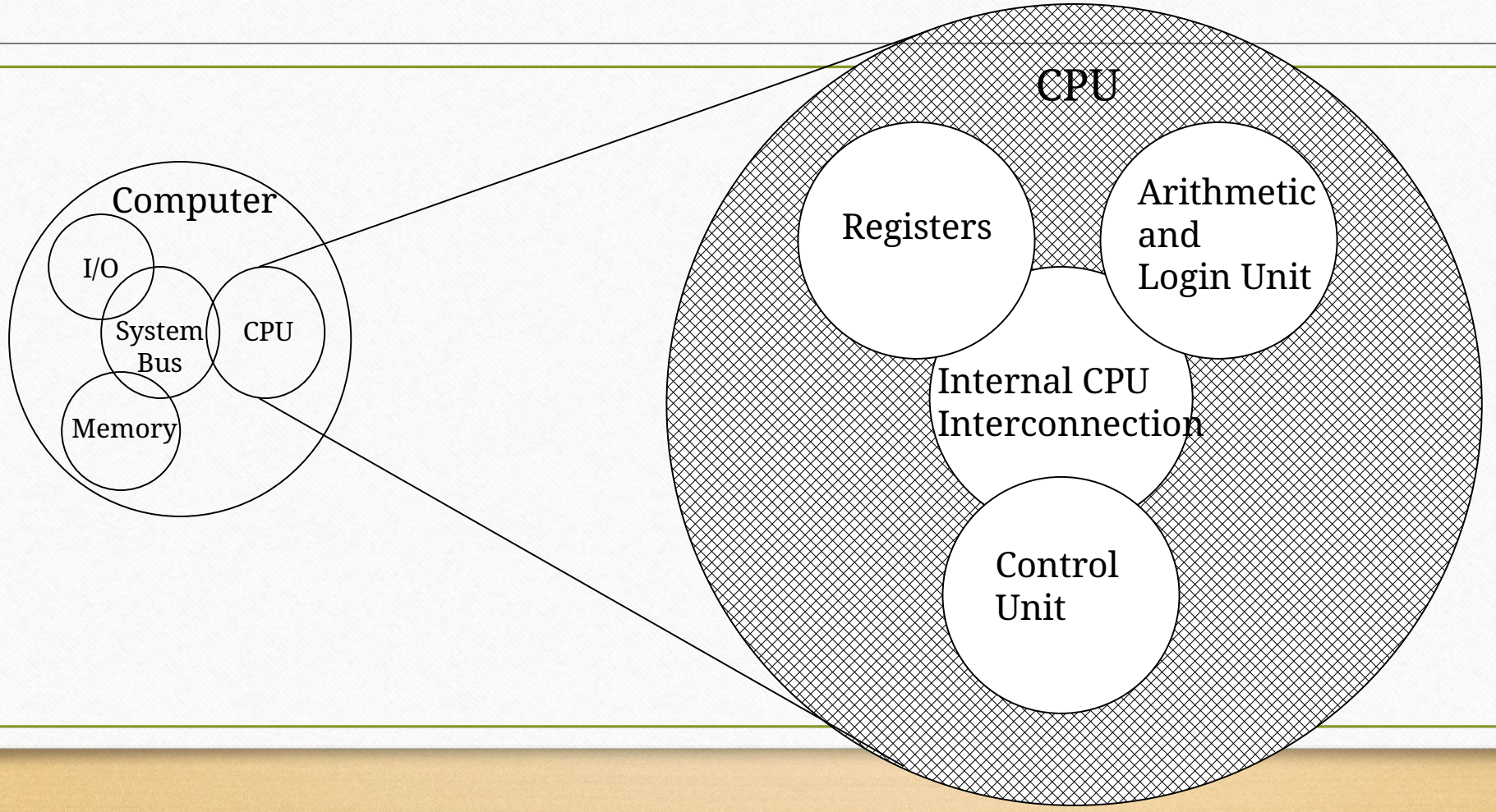
- Processor-Memory-I/O Model
- Components
  - CPU
  - Memory
  - I/O Devices
- Data Flow
- System Interconnect
- Basic Instruction Cycle (Fetch-> Execute)



Structure - Top Level

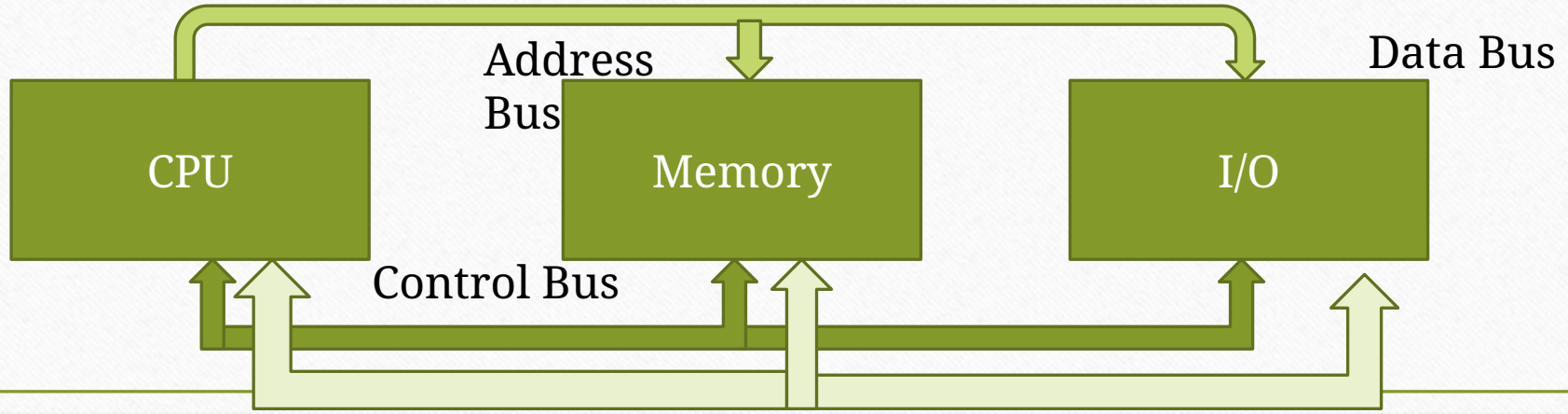


Structure - The CPU



What is a Bus?

- A communication pathway connecting two or more devices
- Often grouped
  - A number of channels in one bus
  - e.g. 32 bit data bus is 32 separate single bit channels



Data Bus

- Carries data
  - Remember that there is no difference between “data” and “instruction” at this level
- Width is a key determinant of performance
  - 8, 16, 32, 64 bit

Bidirectional

Address bus

- Identify the source or destination of data
  - e.g. CPU needs to read an instruction (data) from a given location in memory
- Bus width determines maximum memory capacity of system
  - e.g. 8080 has 16 bit address bus giving 64k address space

Control Bus

- Control and timing information
  - Memory read/write signal
  - Interrupt request
  - Timing signals

Programming Languages

- High-Level Languages (HLL)
- Assembly Language
- Machine Language

High-Level Language

- Allow programmers to write programs that look more like natural language.
- Examples: C++, Java, C#, .NET etc
- A program called **Compiler** is needed to translate a high-level language program into machine code.
- Each statement usually translates into multiple machine language instructions.

Machine Language

- The "native" language of the computer
- Numeric instructions and operands that can be stored in memory and are directly executed by computer system.
- Each ML instruction contains an *op code* (operation code) and zero or more operands.
- Examples:

Opcode Operand Meaning

----

40 increment the AX register

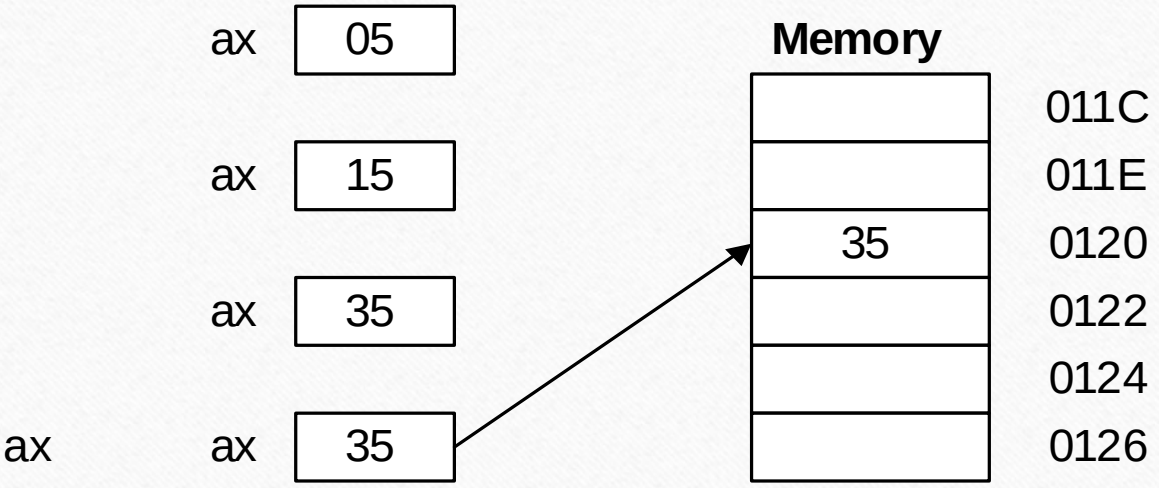
05 0005 add 0005 to AX

Assembly Language

- Use instruction mnemonics that have one-to-one correspondence with machine language.
- An ***instruction*** is a symbolic representation of a single machine instruction
- Consists of:
  - label always optional
  - mnemonic always required
  - operand(s) required by some instructions
  - comment always optional

Sample Program

1. mov ax, 5
2. add ax, 10
3. add ax, 20
4. mov [0120], ax
5. int 20



CPU

- Brain of Computer; controls all operations
  - Uses Memory Circuits to store information
  - Uses I/O Circuits to communicate with I/O Devices
- Executes programs stored in memory
  - System programs
  - Application programs
- **Instruction Set:** Instructions performed by CPU
- Two main components:
  - Execution Unit (EU)
  - Bus Interface Unit (BIU)

Intel 8086 Microprocessor

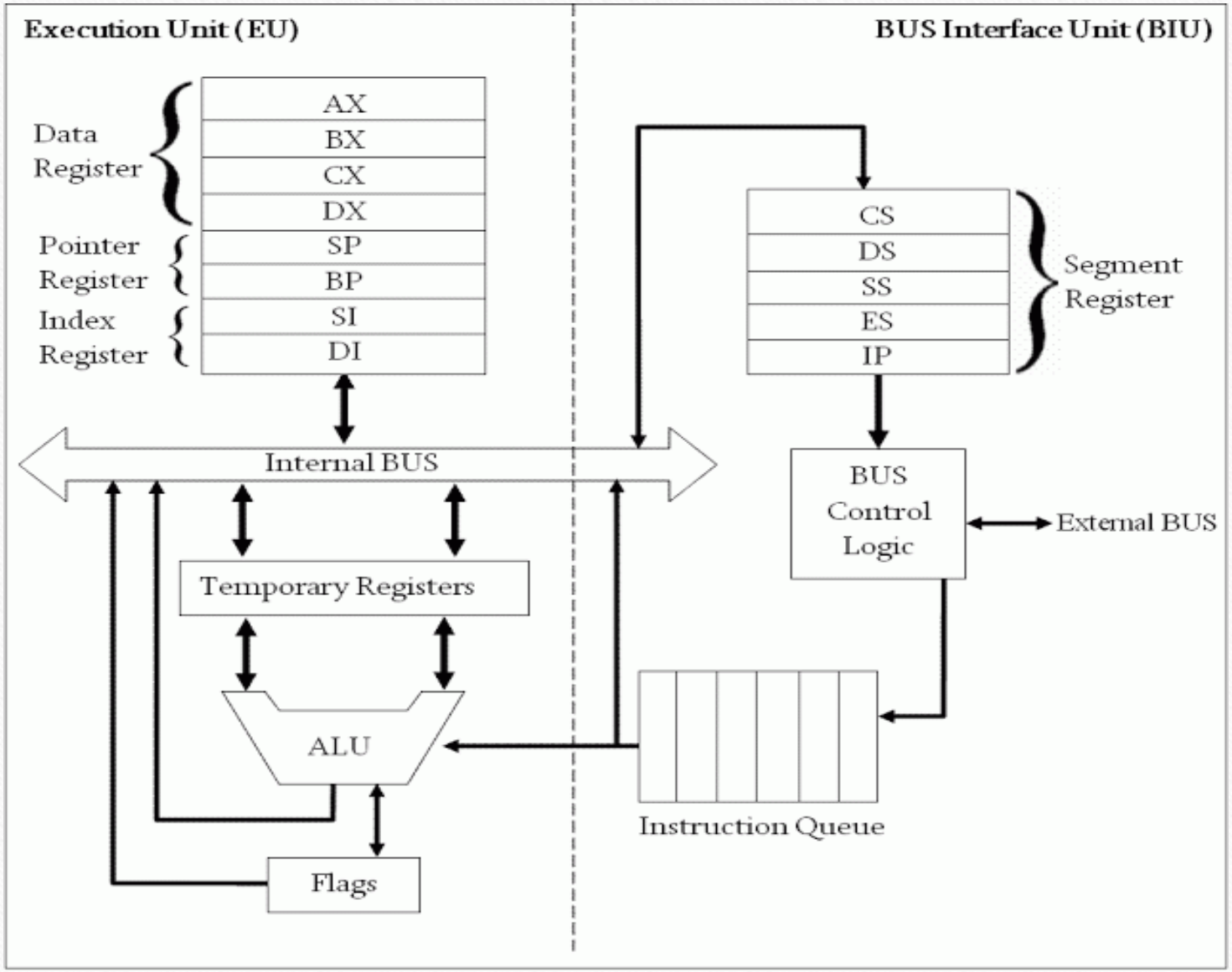
- Introduced in 1978
- One of the first 16-bit microprocessors.
- It laid the foundation for the x86 architecture, which remains prevalent today.

Execution Unit (EU)

- Purpose: Execute instructions
- Contains ALU (Arithmetic & Logic Unit)
  - To perform arithmetic (+, -, x,/) and logic (AND, OR, NOT) operations.
- The data for the operations are stored in circuits called **Registers**.
- A register is like a memory location except that it is referred by a name not a number (address).
- EU uses registers for:
  - Storing data.
  - Holding operands for ALU
  - To reflect result of a computation – FLAG register

Bus Interface Unit (BIU)

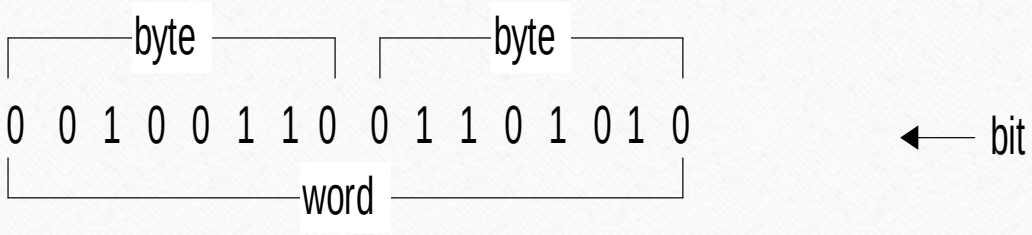
- Facilitates communication between the EU and the memory or I/O circuits.
- Responsible for transmitting address, data and control signals on the buses.



Intel 8086 Microprocessor Organization

Bits, Bytes and Double words

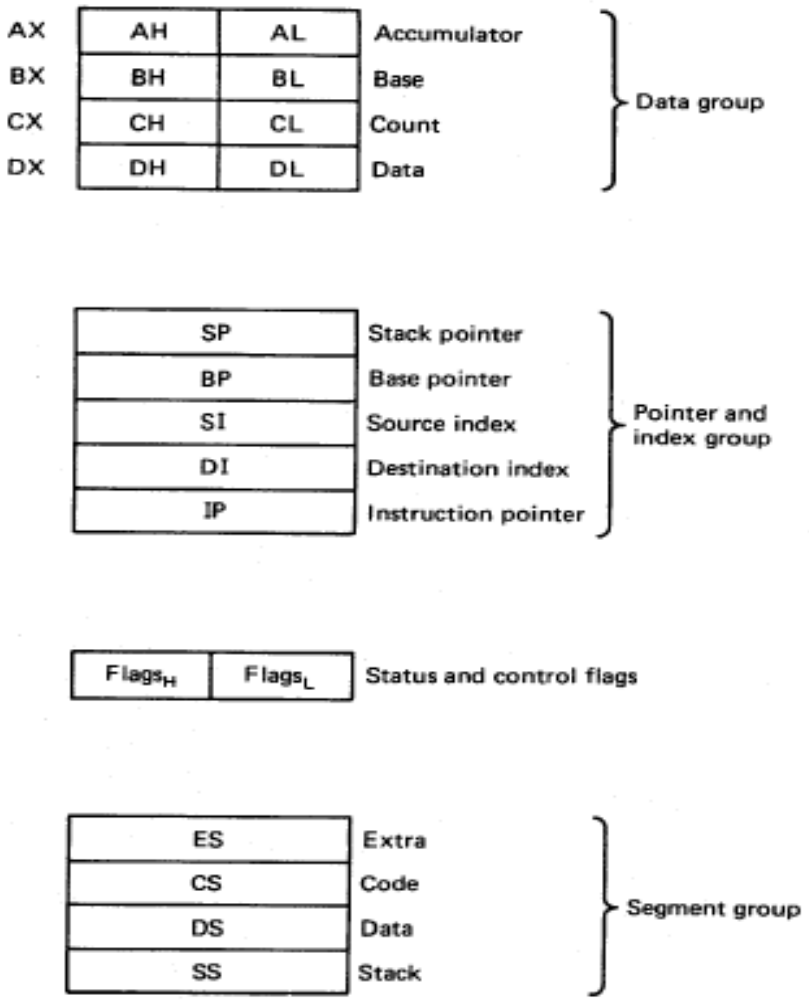
- Each 1 or 0 is called a *bit*.
- Group of 8 bits = **B**yte
- Group of 16 bits = **W**ord
- Group of 32 bits = **D**ouble words



Registers

- Registers are high-speed storage locations inside the microprocessor.
- Designed to be accessed at much higher speed than conventional memory.
- Registers are classified according to the functions they perform.
- General Types of Registers:
  - **Data Registers**: To hold data for an operation.
  - **Address Registers**: To hold the address of an instruction or data.
  - **Status/Flag Register**: keeps the current status of the processor or result of an arithmetic operation.

8086 Internal registers 16 bits (2 bytes ea



AX, BX, CX and DX are two bytes wide and each byte can be accessed separately

These registers are used as memory *pointers*.

6 status; 3 control ; 7 unused

Segment registers are used as base address for a segment

General Purpose/Data Registers

- Following four registers are available to the programmer for general data manipulation:
- **AX (Accumulator):** Used in arithmetic, logic and data transfer instructions. Also required in multiplication, division and input/output operations.
- **BX (Base):** It can hold a memory address that points to a variable.
- **CX (Counter):** Act as a counter for repeating or looping instructions. These instructions automatically repeat and decrement CX and quit when equals to 0.
- **DX (Data):** It has a special role in multiply and divide operations. Also used in input/output operations.

Segment Registers

- Store addresses of instruction and data in memory.
- These values are used by the processor to access memory locations.
- **CS (Code)**: Defines the starting address of the section of memory holding code.
- **DS (Data)**: Defines the section of memory that holds most of the data used by programs.
- **ES (Extra)**: This is an additional data segment that is used by some of the string instructions.
- **SS (Stack)**: It defines the area of the memory used for stack

Pointers and Index Registers

- **IP - instruction pointer**: Always points to next instruction to be executed. IP register always works together with CS segment register.
- **SI - source index register**: Can be used for pointer addressing of data. Works together with DS segment register.
- **DI - destination index register**: Can be used for pointer addressing of data. Works together with ES segment register.
- **BP - base pointer**: Primarily used to access parameters passed via the stack. Works together with SS segment register.
- **SP – stack pointer**: Always points to top item on the stack. Works together with SS segment register.

Thank You